

- S01.04 - Mini project - Create, Validate, Import, Explore
 - Evaluation
 - Tutorial Document
 - Dataset Selection
 - Application Design
 - Preparation Phase
 - Install MongoDB and mongoimport
 - Schema Analysis and Design
 - Import Planning
 - Indexing Strategy
 - Import Process
 - Verification and Validation
 - Summary and Reflection
- Importing data in MongoDB
 - In mongosh with `insertMany()`
 - Using `mongoimport` command line

S01.04 - Mini project - Create, Validate, Import, Explore

In this mini project you will create a new MongoDB database from a **dataset of your choice**.

Your task consist in - Choosing an interesting dataset - Importing the dataset into a MongoDB app - Writing about the whole process

You can work solo or in groups of 2 max.

Evaluation

Please fill in the [spreadsheet](#) with a link to the dataset, and a link to the google doc that you will submit.

- Please use google doc
- invite alexis.perrier@gmail.com as editor

This mini project is graded.

You will have 5' to present your work on June 2nd. - no slides needed, just screen sharing and talk

Tutorial Document

Throughout this mini project you will write a tutorial based on your work and experience.

The document must include the following elements:

- justify the dataset choice and its interest
- describe the application that will use it, thinking especially about how the application will exploit the data
- why choose MongoDB rather than a SQL database
- how the data schema choice aligns with data exploitation by the application
- hardware context
- tools used

Throughout the project, - bugs, problems, and solutions found - performance

and

- an exploration of the dataset with queries used in the application

The document must be a redacted Google document.

Invite me as an editor alexis.perrier@gmail.com

If (hahaha) you use LLMs to write, it's ok but please keep it short and read the text before submitting.

As the bard said: **Brevity is the soul of Wit**



Dataset Selection

The dataset choice is free. The only condition is that it must be fairly large, several GB of data

- Kaggle datasets
 - [Open Food Facts](#)
 - [Arxiv](#)
 - [Disasters](#)
- [Awesome Spacecraft Engineering Datasets](#)
- [open data paris](#),
- [data.gouv.fr](#),
- [ademe.fr](#)
- [huggingface](#)

If your dataset is mainly composed of images, sounds, videos, etc., you will need to find a cloud storage solution to avoid storing images directly in MongoDB documents.

The dataset can include long texts (articles, books, ...), IoT data, etc ...

Application Design

Start by choosing the dataset and imagining the use of this data by an application.

For example, if we use the data from the 210k trees of Paris:

- field application for recording tree dimensions
- exploration application for Paris trees for botanists or tourists
- tree management application dedicated to tree type selection and geolocation

When you have an idea of your application, think about how it will consume the data.

What will be the most frequent operations and queries in the application.

For example for the first "field" application

- find a tree based on its geolocation and species
- record a new measurement of a given tree's dimensions
- visualize tree evolution since planting and compare with other trees of the same species

etc...

Preparation Phase

Prepare your working environment.

Note the technical characteristics of your computer that will influence your import strategy.

Note for example:

- The amount of available RAM
- Free disk space
- The type and speed of your processor
- Your operating system (Windows 11 or MacOS)

Install MongoDB and mongoimport

Then install MongoDB Community Edition.

installing

On Windows 11, you will use the MSI installer available on the official website.

On MacOS, you can use Homebrew with the appropriate command.

Verify that the installation is successful by starting the MongoDB server and connecting to it.

To check if `mongoimport` is already installed on your local, do `mongoimport --version`. If it returns the version you're fine, otherwise you must install it.

See this page to download [command line utilities](#). The list of utilities is [here](#). It includes `mongoimport`, `mongoexport`. see also `mongostat` and `mongotop` as diagnostic tools.

Schema Analysis and Design

Before starting the import, examine the original CSV files.

In your documentation, describe:

- The structure of the present data
- Data types for each column
- Potential relationships between different data
- Potential data quality problems: missing values

From this analysis, design your schema validation rules.

Think about the level of validation you want to implement and justify your choice in your documentation.

Import Planning

In your logbook, note:

- The total size of data to import
- The chosen import strategy (bulk or incremental import)
- The reasons for your choice
- Potential risks and workaround solutions considered

Indexing Strategy

Identify fields that will be frequently queried in your requests.

In your documentation, specify:

- The indexes you plan to create
- The chosen time for their creation (before, during or after import)
- The justification for your indexing strategy

Import Process

Start by testing your approach with a small sample of data.

Monitor system resources during import and note:

- Memory consumption
- Processor usage
- Import time
- Any error messages or warnings

If you encounter memory constraints, adjust the import batch size or use the `--numInsertionWorkers` option of `mongoimport`.

Verification and Validation

After import, verify the integrity of your data.

Include:

- Queries used to verify data
- Total number of imported records
- Schema validation test results
- Query performance with your indexes

Summary and Reflection

Your tutorial / journal can include

- A chronological summary of completed steps
- Difficulties encountered and solutions found
- Observations on system performance
- A critical analysis of your approach
- Improvement suggestions for future import
- Lessons learned during this process

Don't forget that errors and difficulties are part of the learning process.

Importing data in MongoDB

We have several options to load the data from a JSON file.

Either in mongosh with `insertMany()` or from the command line with `mongoimport`.

In mongosh with `insertMany()`

The following script

- loads the data from the `trees.json` file
- Insert data into the `trees` collection with `insertMany()`
- times the operation

```
// load the JSON data
const fs = require("fs");
const dataPath = "./trees_1k.json"
const treesData = JSON.parse(fs.readFileSync(dataPath, "utf8"));

// Insert data into the desired collection
let startTime = new Date()
db.trees.insertMany(treesData);
let endTime = new Date()
print(`Operation took ${endTime - startTime} milliseconds`)
```

JavaScript

Using `mongoimport` command line

`mongoimport` is usually the fastest option for large volumes.

By default `mongoimport` takes a ndjson file (one document per line) as input.

But you can also use a JSON file (an array of documents) if you add the flag `--jsonArray`.

The overall syntax for `mongoimport` follows:

```
mongoimport --uri="mongodb+srv://<username>:<password>@<cluster-url>" \
--db <database_name> \
--collection <collection_name> \
--file <path to ndjson file>
```

Bash

Here are other interesting, self explanatory, flags that may come in handy:

- `--mode=[insert|upsert|merge|delete]`
- `--stopOnError`
- `--drop` (drops the collection first)
- `--stopOnError`

In our context, here is a version of the command line, using the `MONGO_ATLAS_URI` environment variable and loading the JSON file `trees_1k.json` in the current folder.

```
time mongoimport --uri="${MONGO_ATLAS_URI}" \
--db treesdb \
--collection trees \
--jsonArray \
--file ./trees_1k.json
```

Bash

which results in

```
Bash
2024-12-13T11:41:45.941+0100 connected to: mongodb+srv://[**REDACTED**]@skatai.v
2024-12-13T11:41:48.942+0100 [#####] treesdb.trees 558KB/55
2024-12-13T11:41:52.087+0100 [#####] treesdb.trees 558KB/55
2024-12-13T11:41:52.087+0100 1000 document(s) imported successfully. 0 document(
mongoimport --uri="${MONGO_ATLAS_URI}" --db treesdb --collection trees --fil 0
```